

Solution 5: Parallel Computing and Simulations

Benjamin Smith

June 22, 2026

Part 1: Simulations on Cauchy

Suppose $\mathbf{X} = (X_1, \dots, X_n)$ is an i.i.d. sample from the shifted Cauchy distribution with density

$$f(x | \theta) = \frac{1}{\pi (1 + (x - \theta)^2)}, \quad x \in \mathbb{R}$$

Our goal is to compare the following 4 estimators of the parameter θ .

- Sample mean

$$\hat{\theta}_n^{(1)} = \bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$$

- Sample median

$$\hat{\theta}_n^{(2)} = M_n = \frac{1}{2} (X_{(k)} + X_{(k+1)})$$

- Modified sample mean

$$\hat{\theta}_n^{(3)} = M_n + \frac{2}{n} \cdot \frac{\partial \ell}{\partial \theta} \Big|_{\theta=M_n}$$

where ℓ is the log-likelihood function.

- Maximum likelihood estimator (MLE) $\hat{\theta}_n^{(4)}$ defined by

$$\ell \left(\hat{\theta}_n^{(4)} | \mathbf{X} \right) = \max_{\theta \in \mathbb{R}} \ell(\theta | \mathbf{X})$$

where ℓ is the log-likelihood function.

1. Derive the likelihood function and log-likelihood function.
2. Simulate data from Cauchy distribution with location 5, and scale 1.
3. Choose your number of simulations.
4. Verify consistency of the estimators.
5. Calculate the mean square error of the estimators.
6. Calculate the coverage probability of the estimators. Calculate $\mathbf{P}_\theta \left(\left| \hat{\theta}_n - \theta \right| \leq \varepsilon \right)$, for $\varepsilon = 0.1$, and $\theta = 5$.

Solution

```

# Simulate Cauchy.
n <- 1000
set.seed(123456)
cauchy_samples <- rcauchy(n, location = 5, scale = 1)

# Function to calculate the derivative of the log-likelihood.
dloglik <- function(x, theta) {
  d1 <- x - theta
  d2 <- 1 + (x - theta)^2
  return(2*sum(d1/d2))
}

# Function to calculate the log-likelihood.
loglik <- function(theta) {
  x <- cauchy_samples
  n <- length(x)
  l1 <- -n*log(pi)
  l2 <- -sum(log(1 + (x - theta)^2))
  return(l1 + l2)
}

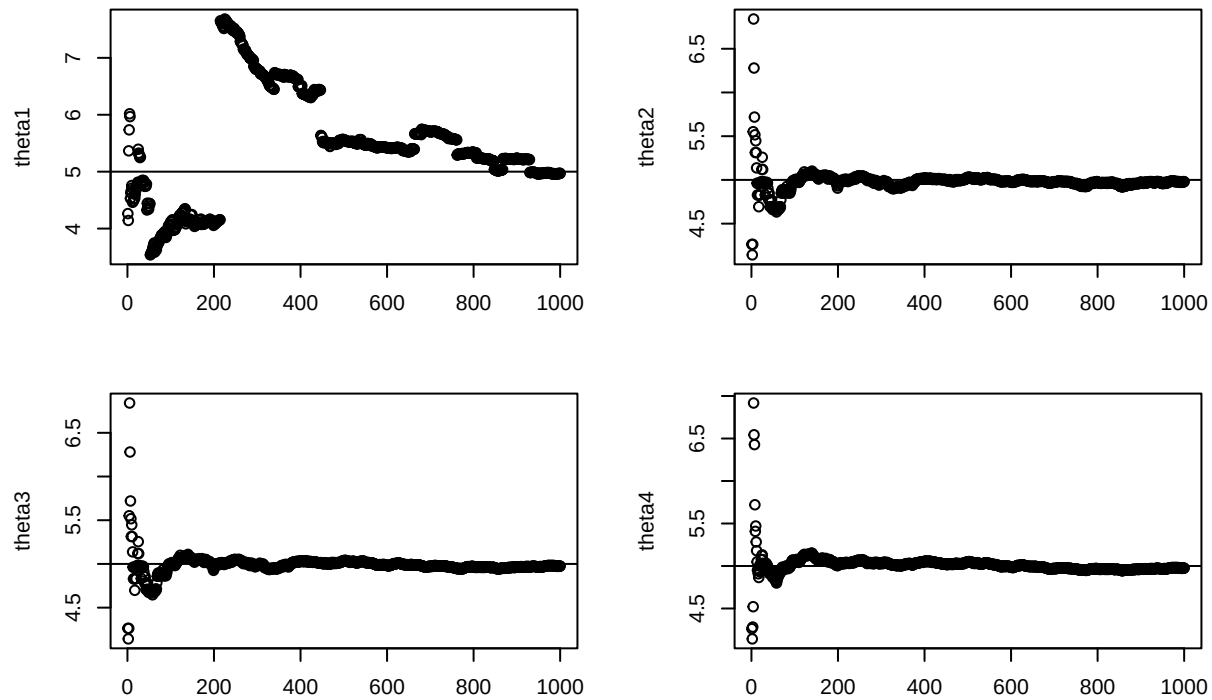
nloglik <- function(theta) {
  return(-loglik(theta))
}

# Consistency.
theta1 <- vector(length = n)
theta2 <- vector(length = n)
theta3 <- vector(length = n)
theta4 <- vector(length = n)

full_samples <- cauchy_samples
for (k in 1:n) {
  cauchy_samples <- NULL
  cauchy_samples <- full_samples[1:k]
  #print(cauchy_samples)
  theta1[k] <- mean(cauchy_samples)
  #print(theta1[i])
  theta2[k] <- median(cauchy_samples)
  theta3[k] <- theta2[k] + 2/n * dloglik(cauchy_samples, theta2[k])
  theta4[k] <- optimize(nloglik, interval = c(-10,10))$minimum
}

par(mfrow = c(4, 2), mar = c(1, 4.1, 4.1, 2.1))
plot(theta1)
abline(h = 5)
plot(theta2)
abline(h = 5)
plot(theta3)
abline(h = 5)
plot(theta4)
abline(h = 5)

```



```
nsim <- 1000
theta1 <- vector(length = nsim)
theta2 <- vector(length = nsim)
theta3 <- vector(length = nsim)
theta4 <- vector(length = nsim)

for (i in 1:nsim) {
  set.seed(1234 + i)

  n <- 1000
  cauchy_samples <- rcauchy(n, location = 5, scale = 1)

  theta1[i] <- mean(cauchy_samples)
  theta2[i] <- median(cauchy_samples)
  theta3[i] <- theta2[i] + 2/n * dloglik(cauchy_samples, theta2[i])
  theta4[i] <- optimize(nloglik, interval = c(-10,10))$minimum
}
```

```
mse <- function(hat_theta, theta) {
  return(mean(hat_theta - theta)^2)
}
```

```
cat("mse1 =", mse(theta1, 5), "\n")
```

```
## mse1 = 1.530928
```

```
cat("mse2 =", mse(theta2, 5), "\n")
```

```
## mse2 = 6.172925e-06
```

```

cat("mse3 =", mse(theta3, 5), "\n")

## mse3 = 1.968817e-06

cat("mse4 =", mse(theta4, 5), "\n")

## mse4 = 1.835778e-06

# Coverage probability
cov_prob <- function(hat_theta, theta, eps = 0.1) {
  return(mean(abs(hat_theta - theta) <= eps))
}

cat("cov_prob1 =", cov_prob(theta1, 5), "\n")

## cov_prob1 = 0.064

cat("cov_prob2 =", cov_prob(theta2, 5), "\n")

## cov_prob2 = 0.957

cat("cov_prob3 =", cov_prob(theta3, 5), "\n")

## cov_prob3 = 0.972

cat("cov_prob4 =", cov_prob(theta4, 5), "\n")

## cov_prob4 = 0.973

```

Part 2: Bootstrap

Applying bootstrap

The following code generates (X_i, Y_i) pairs.

```

library(MASS)

## Warning: package 'MASS' was built under R version 4.4.3

generate_pairs <- function(n) {
  # Generate n pairs of financial returns.
  muX <- 2
  muY <- -1
  CovMx <- matrix(c(1, -.25, -.25, 2), nrow = 2)
  data <- mvrnorm(n = 100, mu = c(muX, muY), Sigma = CovMx)
  return(data.frame('X' = data[, 1],
                    'Y' = data[, 2]))
}

```

```
fin_pairs <- generate_pairs(100) # Generate 100 (X,Y) pairs.
head(fin_pairs)
```

```
##           X           Y
## 1 2.071835  0.8430176
## 2 1.848234 -1.6424359
## 3 4.113012 -1.9049330
## 4 1.737392 -0.9918220
## 5 2.779548 -0.4482892
## 6 3.250761  0.5000100
```

We are interested in

$$\hat{\alpha} = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}$$

```
Sigmahat <- cov(fin_pairs)
Sigmahat
```

```
##           X           Y
## X  1.0446528 -0.3185646
## Y -0.3185646  2.6500412
```

```
sigma2hatXX <- Sigmahat[1,1]
sigma2hatYY <- Sigmahat[2,2]
sigmahatXY <- Sigmahat[1,2]
```

The $\hat{\alpha}$ is

```
alphahat <- (sigma2hatYY - sigmahatXY)/(sigma2hatXX + sigma2hatYY -2*sigmahatXY)
alphahat
```

```
## [1] 0.6853017
```

While the true value of α is

```
sigma2XX <- 1
sigma2YY <- 2
sigmaXY <- -0.25
alpha_true <-(sigma2YY - sigmaXY)/(sigma2XX + sigma2YY -2*sigmaXY)
alpha_true
```

```
## [1] 0.6428571
```

Now, again, we're going to resample with replacement from our data, and compute our statistic $\hat{\alpha}$ on each resample. The hope is that these resampled versions of the statistic will resemble the distribution of the statistic evaluated on the original data.

1. Create a function to compute α from a given data set.
2. Resample the data $B = 200$ times, evaluating $\hat{\alpha}$ on each resample. Then, we'll use those resampled values to estimate the variance.

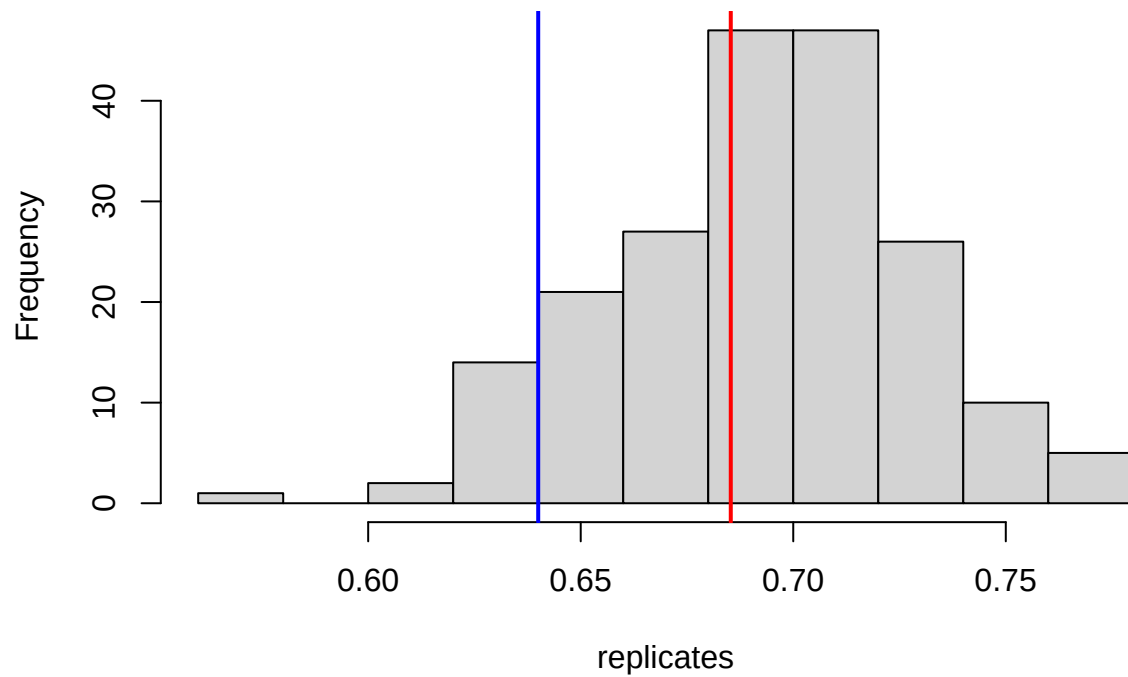
3. Create the confidence interval at the estimate.

Solution

```
compute_alphahat <- function(data) {  
  # We're assuming that data is a data frame with two columns.  
  Sigmahat <- cov( data )  
  # Extract the variance and covariance estimates from the sample covariance  
  sigma2hatXX <- Sigmahat[1,1]  
  sigma2hatYY <- Sigmahat[2,2]  
  sigma2hatXY <- Sigmahat[1,2]  
  # plug these into the definition of alpha.  
  alphahat <- (sigma2hatYY - sigma2hatXY)/(sigma2hatXX + sigma2hatYY -2*sigma2hatXY)  
  return(alphahat)  
}  
  
alphahat <- compute_alphahat(fin_pairs)
```

```
B <- 200  
  
replicates <- rep(NA, B)  
  
# number of observations in our data set.  
n <- nrow( fin_pairs )  
  
for( i in 1:B ) {  
  # To resample the data, we will sample indices, and then grab those rows.  
  resample_indices <- sample(1:n, n, replace=TRUE )  
  resampled_data <- fin_pairs[resample_indices, ]  
  replicates[i] <- compute_alphahat(resampled_data)  
}  
  
hist(replicates)  
abline(v=alphahat, col='red', lwd=2) # alpha of true data.  
abline(v=0.64, col='blue', lwd=2) # True alpha
```

Histogram of replicates



```
# Estimate the variance of alphahat from our bootstrap replicates.  
sd_alphahat <- sd(replicates) # estimate of the std dev of alphahat  
CI <- c(alphahat - 1.96*sd_alphahat, alphahat + 1.96*sd_alphahat)  
CI
```

```
## [1] 0.6176503 0.7529531
```